
SN360 TP1

Introduction.....	3
T.M : Travail Maison.....	3
1.Codage d'un compteur modulo 16.....	3
1.1 Code VHDL RTL.....	3
1.2 Testbench.....	5
1.3 Chronogrammes.....	6

Introduction

Dans le cadre de la matière SN360, nous présentons ce compte-rendu qui retrace les étapes de conception et de mise en œuvre d'un système séquentiel numérique. Dans une première phase théorique, nous avons conçu l'architecture VHDL RTL d'un compteur modulo 16 sur 4 bits intégrant un reset synchrone, une horloge et un signal d'activation enable. Nous avons également prédit son comportement par le tracé du chronogramme du testbench. Dans la phase pratique, nous mobilisons l'outil Vivado pour valider nos modèles par la simulation fonctionnelle avant de procéder à l'implémentation physique. Nous finalisons notre étude par la programmation d'une cible FPGA Artix-7 sur carte Basys3, nous permettant ainsi de vérifier concrètement le fonctionnement d'un compteur modulo 8 via un afficheur 7 segments.

T.M : Travail Maison

1.Codage d'un compteur modulo 16

1.1 Code VHDL RTL

Nous allons coder en VHDL RTL l'architecture d'un compteur modulo 16 sur 4 bits, en implémentant :

- un reset synchrone actif sur niveau haut
- le comptage sur front montant d'horloge
- l'activation/désactivation du comptage (e.q. enable)
- la gestion automatique du débordement (c'est-à-dire retour à 0 du compteur)
- le témoin de débordement (signal deb)

On cherche donc à intégrer le circuit ci-contre

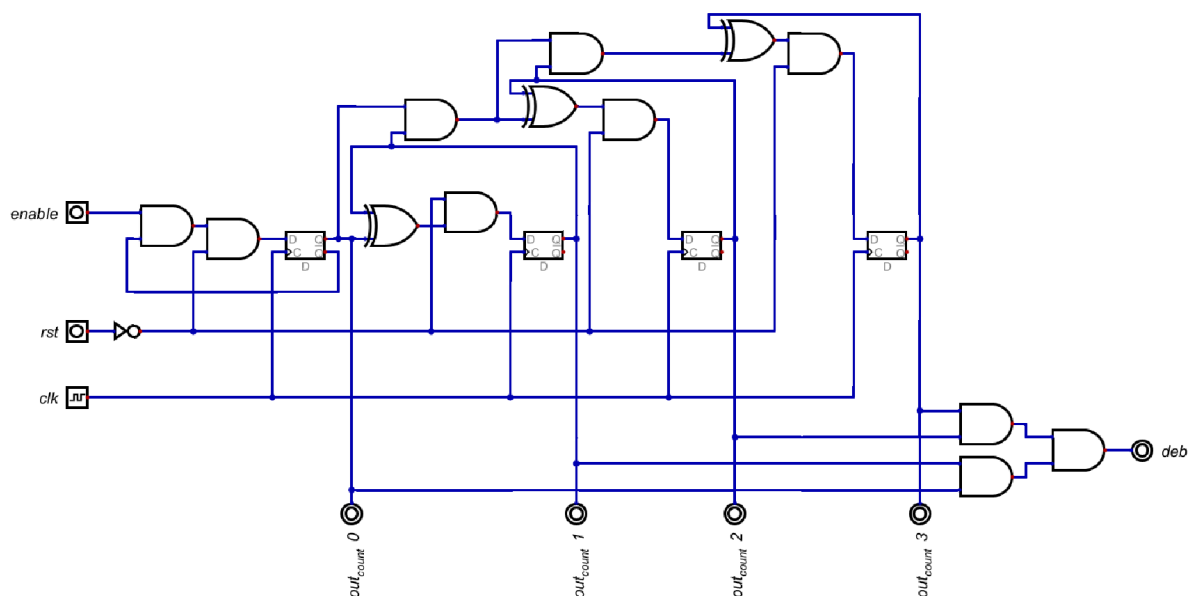


Figure 1.1.1

Code VHDL RTL du compteur :

```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.NUMERIC_STD.ALL;
4
5  entity COUNTER
6  port(
7      clk : in STD_LOGIC;
8      rst : in STD_LOGIC;
9      enable in STD_LOGIC;
10     deb : out STD_LOGIC;
11     out_count : out STD_LOGIC_VECTOR(3 downto 0);
12 );
13 end COUNTER;
14
15 architecture RTL of COUNTER is
16     signal count : UNSIGNED(3 downto 0) :
17 begin
18     process(clk)
19     begin
20         if rising_edge(clk) then
21             if rst = '1' then
22                 count <= (others => '0');
23             elsif enable == '1' then
24                 if count == "1111" then
25                     count <= (others => '0');
26                 else
27                     count <= count + 1;
28                 end if;
29             end if;
30         end if;
31     end process;
32     out_count <= std_logic_vector(count);
33     deb <= '1' when count = "1111" else '0';
34 end RTL;
```

Figure 1.1.2

- Bibliothèques : Nous utilisons NUMERIC_STD pour utiliser le type UNSIGNED, ce qui permet d'effectuer l'opération d'incrément +1.
- Logique séquentielle : Le bloc process(clk) permet le déclenchement uniquement sur le front montant de l'horloge (rising_edge).
- Reset : test du signal rst en premier dans le processus pour garantir un reset prioritaire.
- Comptage : l'incrément ne s'effectue que si le signal d'activation enable est à 1. Le retour à zéro est géré automatiquement après la valeur "1111" pour gérer le cycle modulo 16.
- Signaux de sortie : le signal interne count est converti en std_logic_vector pour la sortie out_count, cela permet d'éviter de manipuler directement out_count, qui est déclaré comme une sortie. Parallèlement, le signal deb passe à 1 si la valeur maximale est atteinte, reste à 0 sinon.

1.2 Testbench

```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3
4  entity COUNTER_TB is
5  end COUNTER_TB;
6
7  architecture bhv of COUNTER_TB is
8      signal clk_TB, rst_TB, enable_TB, deb_TB : STD_LOGIC;
9      signal out_count_TB : STD_LOGIC_VECTOR(3 downto 0);
10
11     -- Ici, on fait le lien entre l'architecture de COUNTER
12     -- et le testbench
13     component COUNTER
14     port(
15         clk : in STD_LOGIC;
16         rst : in STD_LOGIC;
17         enable : in STD_LOGIC;
18         deb : out STD_LOGIC;
19         out_count : out STD_LOGIC_VECTOR(3 downto 0)
20     );
21     end component;
22
23     begin
24         clk_TB <= not clk_TB after 10 ns;
25         rst_TB <= '0', '1' after 30 ns, '0' after 40 ns,
26             '1' after 90 ns, '0' after 110 ns;
27         enable_TB <= '0', '1' after 50 ns, '0' after 70 ns,
28             '1' after 90 ns;
29
30         DUT: COUNTER
31         port map(
32             clk => clk_TB,
33             rst => rst_TB,
34             enable => enable_TB,
35             deb => deb_TB,
36             out_count => out_count_TB
37         );
38
39     end bhv;
```

Figure 1.2.1

- L'entité est laissée vide car un testbench n'a pas d'entrées-sorties physiques. On déclare les signaux internes servant de fils pour injecter les stimuli et observer les résultats.
- Le bloc component COUNTER crée le lien avec l'architecture du compteur, en reprenant les mêmes ports (clk, rst, enable, deb, out_count).
- La ligne `clk_TB <= not clk_TB after 10 ns` génère un front montant toutes les 20 ns.
- Les signaux `rst_TB` et `enable_TB` reçoivent les signaux permettant de valider les différents scénarios de fonctionnement : remise à zéro, maintien et comptage.
- Le bloc port map réalise le branchement entre les signaux de test et les ports du compteur pour permettre la simulation.

1.3 Chronogrammes

En suivant le testbench, on obtient les chronogrammes suivant :

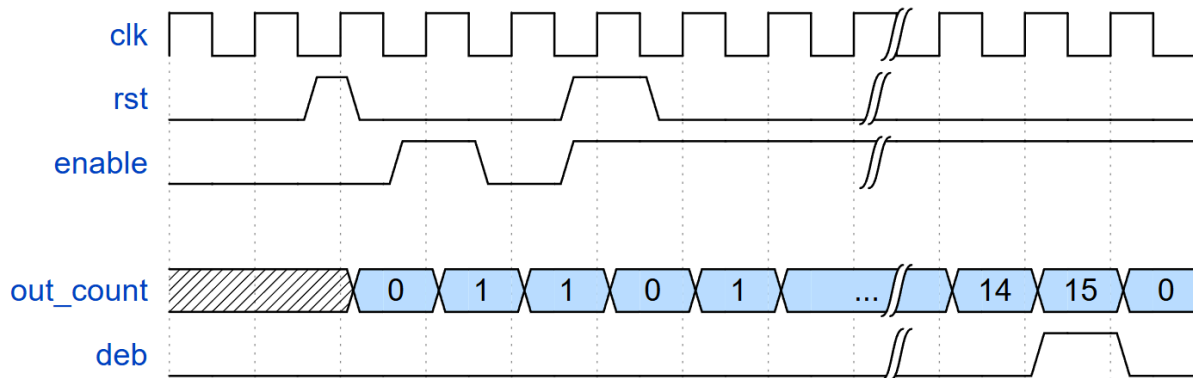


Figure 1.3.1

Les '...' représente ici la valeur de **deb** qui augmente de 1 à chaque front d'horloge.